# LAPORAN PRAKTIKUM
# ALGORITMA & STRUKTUR DATA
# MODUL 4

**Double Linked List**

**Oleh:**

**Ahmad Riyadhi** **NIM. 2510817110004**

**PROGRAM STUDI TEKNOLOGI INFORMASI**

**FAKULTAS TEKNIK**

**UNIVERSITAS LAMBUNG MANGKURAT**

**APRIL 2026**

# LEMBAR PENGESAHAN
# LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
# MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4 : Double Linked List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Ahmad Riyadhi
NIM : 2510817110004

| | |
|---|---|
| Menyetujui,<br>Asisten Praktikum | Mengetahui,<br>Dosen Penanggung Jawab Praktikum |
| Fadhil Syahdama Mahatma Putra<br>NIM. 2410817210026 | Andreyan Rizky Baskara, S.Kom., M.Kom<br>NIP. 199307032019031011 |

# DAFTAR ISI

LEMBAR PENGESAHAN ..... 1
DAFTAR ISI ..... 2
DAFTAR TABEL ..... 3
DAFTAR GAMBAR ..... 4
SOAL 1 ..... 5
Pembahasan ..... 5
SOAL 2 ..... 22
Pembahasan ..... 22
SOAL 3 ..... 23
Pembahasan ..... 23
SOAL 4 ..... 24
Pembahasan ..... 24
TAUTAN GIT HUB ..... 26

# DAFTAR TABEL

Table 1 Source Code Double Linked List ..... 5

# DAFTAR GAMBAR

gambar 1 Tampilan awal program ..... 14
gambar 2 Tampilan program  jika user memilih opsi 1 ..... 14
gambar 3 Tampilan program  jika user memilih opsi 2 ..... 14
gambar 4 Tampilan program  jika user memilih opsi 3 ..... 15
gambar 5 Tampilan awal program soal 4 ..... 24
gambar 6 Tampilan program  jika user meng input 1 2 3 ..... 24
gambar 8 Tampilan program  jika user meng input 2 3 1 ..... 25
gambar 9 Tampilan program  jika user meng input 3 2 1 ..... 25

## SOAL 1

Lengkapi coding pada function tambahDepanH() agar bisa berjalan dengan lancar. running, simpan program, dan screenshoot hasil running !

**Pembahasan**

*Table 1 Source Code Double Linked List*

```
#include <conio.h>
#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct Tnode {
string data;
Tnode *next;
Tnode *prev;
};

Tnode *head, *tail;

int pil, menu;
char pilihan[1];
string dataBaru;

void initH();
void initHT();
int isEmptyH();
int isEmptyHT();

void tambahDepanH();
void tambahDepanHT();
void tambahBelakangH();
void tambahBelakangHT();
void hapusDepanH();
void hapusDepanHT();
void hapusBelakangH();
void hapusBelakangHT();
void tampilkanH();
void tampilkanHT();
void clearH();
```

```
void clearHT();

int main()
{
menu:
cout<<"Double Linked List Non Circular (DLLNC)"<<endl;
cout<<"========================================"<<endl;
cout<<"Silahkan pilihan program DLLNC yang ingin dijalankan!"<<endl;
cout<<"1. DLLNC dengan head"<<endl;
cout<<"2. DLLNC dengan Head dan Tail"<<endl;
cout<<"3. Quit"<<endl;
cout<<"Pilihan"<<endl;
cin>>menu;
system("cls");
if(menu==1){
do {
cout<<"Double Linked List Non Circular (DLLNC) (head)"<<endl;
cout<<"=============================================="<<endl;
cout<<"1. Tambah Depan"<<endl;
cout<<"2. Tambah Belakang"<<endl;
cout<<"3. Tampilkan Data"<<endl;
cout<<"4. Hapus Depan"<<endl;
cout<<"5. Hapus Belakang"<<endl;
cout<<"6. Reset"<<endl;
cout<<"7. Kembali Ke Menu"<<endl;
cout<<"Pilihan : "<<endl;
cin>>pilihan;
pil=atoi(pilihan);

switch (pil){
case 1:
tambahDepanH();
break;
case 2:
tambahBelakangH();
break;
case 3:
tampilkanH();
break;
case 4:
hapusDepanH();
break;
```

```
case 5:
hapusBelakangH();
break;
case 6:
clearH();
break;
default:
system("cls");
goto menu;
}

cout<<"\npress any key to continue"<<endl;
getch();
system("cls");

} while (pil<7);
} else if(menu==2){
do {
cout<<"Double Linked List Non Circular (DLLNC) (head dan Tail)"<<endl;
cout<<"==============================================="<<endl;
cout<<"1. Tambah Depan"<<endl;
cout<<"2. Tambah Belakang"<<endl;
cout<<"3. Tampilkan Data"<<endl;
cout<<"4. Hapus Depan"<<endl;
cout<<"5. Hapus Belakang"<<endl;
cout<<"6. Reset"<<endl;
cout<<"7. Kembali Ke Menu"<<endl;
cout<<"Pilihan : "<<endl;
cin>>pilihan;
pil=atoi(pilihan);

switch (pil){
case 1:
tambahDepanHT();
break;
case 2:
tambahBelakangHT();
break;
case 3:
tampilkanHT();
break;
case 4:
```

```
hapusDepanHT();
break;
case 5:
hapusBelakangHT();
break;
case 6:
clearHT();
break;
default:
system("cls");
goto menu;
}

cout<<"\npress any key to continue"<<endl;
getch();
system("cls");

} while (pil<7);
} else {
cout<<"\nTERIMA KASIH"<<endl;
cout<<"Program was made by Ahmad Riyadhi (2510817110004)"<<endl;
}
}

void initH(){
head=NULL;
}

void initHT(){
head=NULL;
tail=NULL;
}

int isEmptyH(){
if(head == NULL) return 1;
else return 0;
}

int isEmptyHT(){
if(tail == NULL) return 1;
else return 0;
}
```

```

void tambahDepanH() {
cout<<"Masukkan Data : ";
cin>>dataBaru;
Tnode *baru;
baru = new Tnode;
baru->data = dataBaru;
baru->next = NULL;
baru->prev = NULL;
if(isEmptyH() == 1) {    // <-- Pakai isEmptyH()
head = baru;
} else {
baru->next = head;
head->prev = baru;
head = baru;
}
cout<<"Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan.";
}

void tambahDepanHT() {
cout<<"Masukan Data : ";
cin>>dataBaru;
Tnode *baru;
baru = new Tnode;
baru->data = dataBaru;
baru->next = NULL;
baru->prev = NULL;
if(isEmptyHT() == 1) {
head = baru;
tail = baru;
} else {
baru->next = head;
head->prev = baru;
head = baru;
}
cout<<"Data \""<<dataBaru<<"\" berhasil dimasukan di bagian depan.";
}

void tambahBelakangH() {
cout<<"Masukan Data : ";
cin>>dataBaru;
```

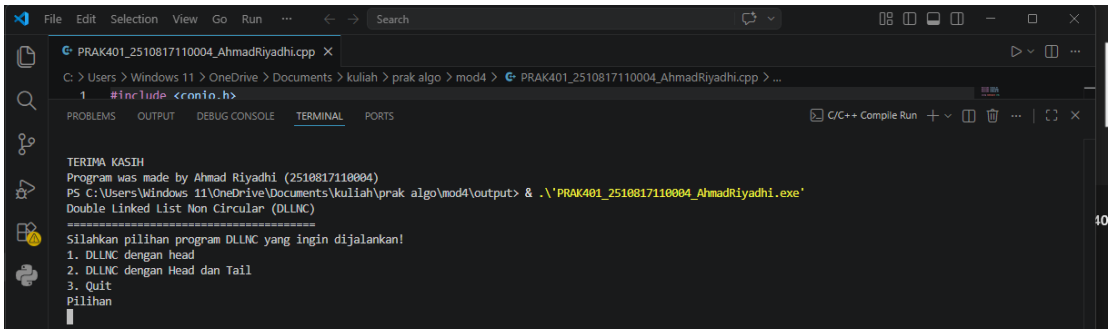
```
Tnode *baru, *bantu;
baru = new Tnode;
baru->data = dataBaru;
baru->next = NULL;
baru->prev = NULL;
if(isEmptyH() == 1) {
head = baru;
} else {
bantu = head;
while(bantu->next != NULL) {
bantu = bantu->next;
}
bantu->next = baru;
baru->prev = bantu;
}
cout <<"Data \""<<dataBaru<<"\" berhasil dimasukan di bagian belakang.";
}

void tambahBelakangHT() {
cout<<"Masukan Data : ";
cin>>dataBaru;
Tnode *baru;
baru = new Tnode;
baru->data = dataBaru;
baru->next = NULL;
baru->prev = NULL;
if(isEmptyHT() == 1) {
head = baru;
tail = baru;
} else {
tail->next = baru;
baru->prev = tail;
tail = baru;
}
cout<<"Data \""<<dataBaru<<"\" berhasil dimasukan di bagian belakang.";
}

void tampilkanH() {
Tnode *bantu;
bantu = head;
if(isEmptyH() == 0) {
```

```
while(bantu != NULL) {
cout<<bantu->data<<' ';
bantu = bantu->next;
}
cout<<endl;
} else cout<<"Tidak terdapat data pada Linked List";
}

void tampilkanHT() {
Tnode *bantu;
bantu = head;
if(isEmptyHT() == 0) {
while(bantu != tail->next) {
cout<<bantu->data<<' ';
bantu = bantu->next;
}
cout<<endl;
} else cout<<"Tidak terdapat data pada Linked List";
}

void hapusDepanH() {
Tnode *hapus;
string data;
if(isEmptyH() == 0) {
hapus = head;
data = hapus->data;
if(head->next != NULL) {
head = head->next;
head->prev = NULL;
} else {
initH();
}
delete hapus;
cout<<"Data \""<<data<<"\" berhasil dihapus dari bagian depan."<<endl;
} else cout<<"Tidak ada data pada Linked List"<<endl;
}

void hapusDepanHT() {
Tnode *hapus;
string data;
if(isEmptyHT() == 0) {
hapus = head;
```

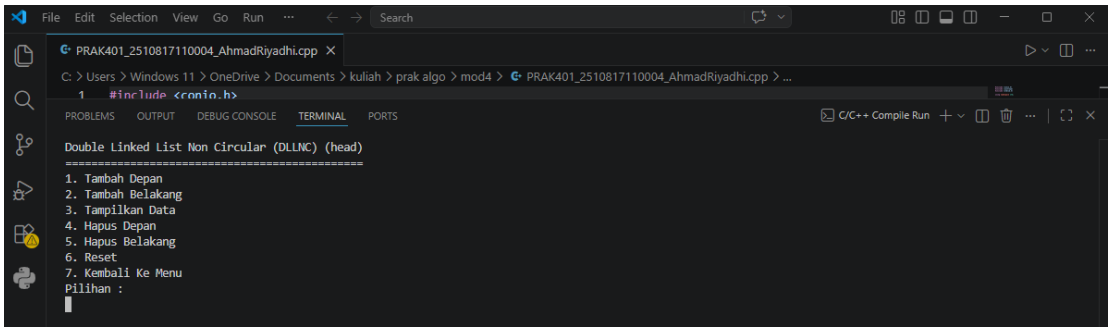
```
data = hapus->data;
if(head->next != NULL) {
head = head->next;
head->prev = NULL;
} else {
initHT();
}
delete hapus;
cout<<"Data  \""<<data<<"\"  berhasil  dihapus  dari  bagian depan."<<endl;
} else cout<<"Tidak ada data pada Linked List"<<endl;
}

void hapusBelakangH() {
Tnode *hapus;
string data;
if(isEmptyH() == 0) {
hapus = head;
while(hapus->next != NULL) {
hapus = hapus->next;
}
data = hapus->data;
if(head->next != NULL) {
hapus->prev->next = NULL;
} else {
initH();
}
delete hapus;
cout<<"Data  \""<<data<<"\"  berhasil  dihapus  dari  bagian belakang."<<endl;
} else cout<<"Tidak ada data pada Linked List"<<endl;
}

void hapusBelakangHT() {
Tnode *hapus;
string data;
if(isEmptyHT() == 0) {
hapus = tail;
data = hapus->data;
if(head->next != NULL) {
tail = tail->prev;
tail->next = NULL;
} else {
```

```
initHT();
}
delete hapus;
cout<<"Data  \""<<data<<"\"  berhasil  dihapus  dari  bagian belakang."<<endl;
} else cout<<"Tidak ada data pada Linked List"<<endl;
}

void clearH() {
Tnode *bantu, *hapus;
bantu = head;
while(bantu != NULL) {
hapus = bantu;
bantu = bantu->next;
delete hapus;
}
initH();
cout<<"Seluruh     data     pada     Linked     List     telah dibersihkan"<<endl;
}

void clearHT() {
Tnode *bantu, *hapus;
bantu = head;
while(bantu != NULL) {
hapus = bantu;
bantu = bantu->next;
delete hapus;
}
initHT();
cout<<"Seluruh     data     pada     Linked     List     telah dibersihkan"<<endl;
}
```

## Pembahasan

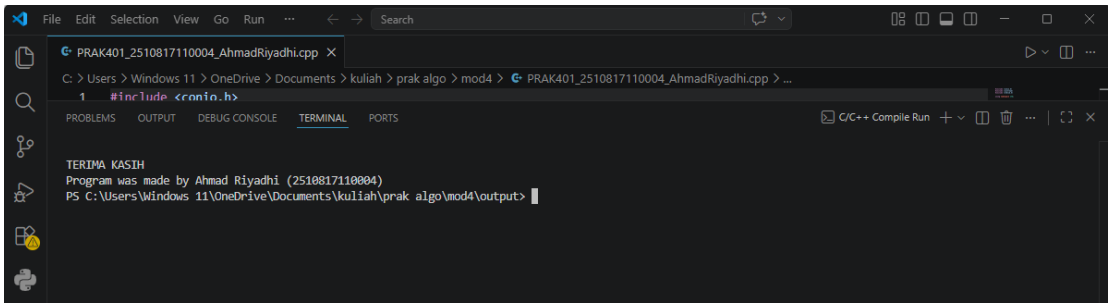



*gambar 1 Tampilan awal program*




*gambar 2 Tampilan program  jika user memilih opsi 1*

*gambar 3 Tampilan program  jika user memilih opsi 2*




*gambar 4 Tampilan program  jika user memilih opsi 3*

**Baris 1 - 3**, syntax #include <conio.h>, #include <iostream>, dan #include <stdlib.h> berfungsi sebagai preprocessor directive untuk memanggil library standar bawaan C++. Library <iostream> secara krusial menangani operasi Input/Output aliran data (seperti cin dan cout), <conio.h> memfasilitasi antarmuka konsol (seperti perintah getch() untuk menahan layar), dan <stdlib.h> menyediakan pustaka utilitas umum, termasuk alokasi memori dinamis, fungsi konversi tipe data seperti atoi(), serta kontrol sistem seperti system("cls") untuk membersihkan buffer layar terminal.

· **Baris 5**, syntax using namespace std; berfungsi untuk mendeklarasikan penggunaan ruang nama standar (standard namespace). Hal ini menginstruksikan kompilator agar secara otomatis mengenali seluruh entitas dalam pustaka standar C++, sehingga mempersingkat penulisan kode secara signifikan karena programmer tidak perlu lagi mengetikkan prefiks scope resolution std:: secara eksplisit dan berulang kali di setiap pemanggilan perintah dasar seperti std::cout, std::cin, atau std::endl.

· **Baris 7 - 11**, syntax typedef struct Tnode berfungsi untuk mendefinisikan sebuah struktur data (user-defined data type) berjenis Double Linked List. Cetakan ini membentuk sebuah objek node (simpul) komprehensif yang di dalamnya dialokasikan tiga ruang memori berbeda: sebuah variabel string data sebagai payload untuk menampung teks aktual, sebuah pointer *next yang menyimpan referensi alamat memori node di depannya (arah maju), serta sebuah pointer *prev yang

menyimpan referensi alamat memori node di belakangnya (arah mundur). Kedua pointer inilah yang memungkinkan navigasi dua arah yang fleksibel.

· **Baris 13**, syntax Tnode *head, *tail; berfungsi untuk mendeklarasikan dua buah pointer global yang tidak menyimpan data, melainkan bertindak secara eksklusif sebagai jangkar atau penanda koordinat mutlak. Pointer head bertugas mengunci posisi elemen paling pertama di dalam untaian list, sedangkan pointer tail bertugas mengunci posisi elemen paling ujung akhir.

· **Baris 15 - 17**, syntax deklarasi variabel bertipe integer (pil, menu), array of character (pilihan[1]), dan string (dataBaru) berfungsi untuk mencadangkan ruang alokasi memori global pada sistem. Variabel-variabel ini diletakkan di luar fungsi agar memiliki global scope (dapat diakses dan dimodifikasi dari dalam blok fungsi mana pun) yang berguna untuk mengelola alur state navigasi menu aplikasi, menampung karakter pilihan menu sementara, serta menyimpan nilai string yang diketikkan pengguna sebelum dimasukkan ke dalam node.

· **Baris 19 - 35**, kumpulan syntax void ...() dan int ...() berfungsi sebagai function prototype atau forward declaration. Deklarasi awal ini sangat penting dalam arsitektur program C++ untuk membangun tabel simbol (symbol table) bagi mesin kompilator. Dengan adanya prototipe ini, kompilator telah mengetahui nama, tipe kembalian, dan parameter dari setiap fungsi sebelum fungsi tersebut benar-benar dipanggil di dalam main(), meskipun rincian logika di dalam fungsi-fungsi tersebut baru dijabarkan secara lengkap di bagian paling bawah source code.

· **Baris 37 - 129**, syntax blok fungsi int main() berfungsi sebagai titik masuk utama (entry point) dan program inti pengendali. Blok ini memuat label antarmuka menu: untuk titik kembali goto, dan menggunakan percabangan kontrol if-else berlapis untuk memisahkan logika eksekusi antara program Linked List yang hanya menggunakan mode Head dengan mode Head & Tail. Di dalam masing-masing blok

percabangan tersebut, disematkan perulangan do-while yang menjamin antarmuka menu akan terus di-render ulang secara berkesinambungan (selama nilai pil < 7). Input dari pengguna ditangkap sebagai array char lalu dikonversi menjadi integer murni menggunakan fungsi atoi() untuk mencegah infinite loop (kerusakan program jika pengguna memasukkan huruf alih-alih angka), yang selanjutnya diproses oleh struktur switch-case guna memicu atau memanggil fungsi-fungsi manipulasi data spesifik (seperti penambahan, penghapusan, atau penampilan) sesuai dengan instruksi (case) yang dipilih pengguna.

· **Baris 132 - 142**, syntax penutup pada blok fungsi main() berisi rangkaian instruksi pembersihan dan terminasi. Perintah getch(); (get character) memaksa utas program untuk berhenti sejenak dan menunggu hingga pengguna memberikan interrupt berupa ketikan sembarang tombol pada keyboard. Setelah ditekan, fungsi system("cls"); akan menghapus seluruh teks statis di antarmuka konsol sebelum siklus menu berikutnya digambar ulang. Percabangan else bertindak sebagai rute terminasi mutlak yang akan mencetak pesan perpisahan beserta identitas hak cipta pembuat program apabila pengguna menekan opsi menu penutup.

· **Baris 144 - 151**, syntax pada blok fungsi initH() dan initHT() (inisialisasi) berfungsi untuk melakukan reset arsitektur memori di awal eksekusi program. Fungsi ini menyetel nilai alamat referensi pointer head dan tail menjadi NULL. Penggunaan NULL secara eksplisit sangat penting untuk memastikan tidak ada dangling pointer (pointer yang menunjuk ke alamat memori acak/sampah) sehingga kompilator dapat mengenali secara pasti bahwa struktur untaian data tersebut berada dalam kondisi kosong.

· **Baris 153 - 162**, syntax pada blok fungsi isEmptyH() dan isEmptyHT() berfungsi sebagai fungsi validasi logika (boolean-like) untuk memeriksa ketersediaan memori data. Fungsi ini mengembalikan nilai integer 1 (merepresentasikan nilai kebenaran

logika True) jika penanda head atau tail ditemukan menunjuk pada ruang hampa (NULL), dan akan mengembalikan nilai 0 (False) jika sebaliknya list tersebut terdeteksi telah memiliki tautan ke struktur memori elemen.

· **Baris 164 - 181**, syntax pada blok fungsi tambahDepanH() berfungsi menangani insersi data secara dinamis di posisi paling awal khusus pada arsitektur mode Head. Prosesnya diawali dengan meminta alokasi blok memori baru pada struktur heap menggunakan perintah new Tnode. Setelah memori baru tersebut diisi dengan beban teks dataBaru, program melakukan proses pointer wiring (penyambungan arah pointer): mengarahkan pointer next elemen baru ke head yang lama, mengarahkan pointer prev elemen lama tersebut kembali ke elemen baru secara menyilang, lalu mendeklarasikan elemen memori baru tersebut secara resmi sebagai titik head terkini.

· **Baris 183 - 200**, syntax pada blok fungsi tambahDepanHT() berfungsi menyisipkan elemen di urutan terdepan pada list berarsitektur Head dan Tail. Secara logika, proses penautan silang pointernya identik dengan tambahDepanH(). Namun perbedaannya, fungsi ini mengintegrasikan pengecekan kondisi khusus: apabila memori tersebut divalidasi kosong saat penyisipan data pertama, maka selain head, pointer tail juga diinstruksikan untuk ikut menunjuk (sync) pada elemen tunggal yang baru diciptakan tersebut agar referensi ekor tidak tertinggal.

· **Baris 202 - 221**, syntax pada blok fungsi tambahBelakangH() berfungsi merangkai memori elemen baru pada posisi paling ujung (tail-end) pada list arsitektur Head. Mengingat list ini tidak memiliki penanda ekor instan, program harus melakukan iterasi dengan kompleksitas waktu O(n). Sebuah pointer pelacak lokal (bantu) ditugaskan untuk menelusuri memori satu per satu dari arah head menuju ke arah belakang melalui siklus perulangan penjelajahan while(bantu->next != NULL). Setelah loop berhenti tepat di elemen terakhir, elemen baru dihubungkan secara bilateral dengan elemen terakhir tersebut.

· **Baris 223 - 237**, syntax pada blok fungsi tambahBelakangHT() berfungsi memproses penambahan beban elemen di titik paling akhir list secara komputasi yang sangat efisien dan instan (kompleksitas waktu O(1)). Karena list ini dilengkapi variabel tail, program sepenuhnya mem- bypass kebutuhan iterasi atau penelusuran ulang dari arah depan. Pointer arah next dari tail lama secara langsung disuntikkan referensi memori node baru, lalu label tail itu sendiri dipindahkan ke objek memori yang paling baru tersebut.

· **Baris 239 - 249**, syntax pada blok fungsi tampilkanH() berfungsi mengekstraksi dan mencetak seluruh data terekam di layar terminal secara sekuensial. Sebuah pointer lokal bernama bantu diposisikan pada node head, lalu dikendalikan oleh siklus perulangan while(bantu != NULL). Di setiap iterasi, program merilis beban string dari pointer bantu->data menuju saluran output konsol, lalu pointer tersebut dipaksa melompat dan memperbarui lokasinya dengan membaca alamat memori yang ada pada bantu->next hingga menyentuh batas NULL.

· **Baris 251 - 261**, syntax pada blok fungsi tampilkanHT() berfungsi mencetak rentetan data secara kronologis dengan fondasi logika penelusuran berulang yang hampir identik. Pembeda signifikannya terletak pada kondisi terminasi loop yang dirancang menjadi while(bantu != tail->next). Pengecekan berbasis tail ini memastikan bahwa siklus pembacaan dan pencetakan data akan dihentikan secara presisi seketika pointer penelusur melampaui posisi penanda ekor.

· **Baris 262 - 278**, syntax pada blok fungsi hapusDepanH() berfungsi memutilasi dan menghancurkan memori elemen di urutan pertama secara aman. Data dari elemen sasaran dicadangkan sementara ke dalam variabel string. Label pangkal head kemudian diinstruksikan untuk bergeser satu langkah ke elemen memori berikutnya (head->next). Untuk memutuskan hubungan dengan elemen yang lama, referensi head->prev dibuat menjadi NULL. Sebagai tindakan fundamental penanggulangan

kebocoran memori komputer (memory leak), objek memori elemen pertama yang telah terisolasi tersebut dihancurkan sepenuhnya menggunakan perintah standar C++ delete.

· **Baris 280 - 296**, syntax pada blok fungsi hapusDepanHT() berfungsi memusnahkan elemen node pertama berserta kapabilitas rilis alokasi memorinya pada list ganda. Alur instruksi pemotongannya serupa, namun diperketat dengan penanganan kasus edge condition yang mutlak: apabila elemen yang baru saja dihancurkan merupakan satu-satunya node sisa yang ada (diketahui melalui head->next == NULL yang bernilai keliru pada fungsi else), maka program memanggil initHT() untuk mengembalikan pointer statis head dan tail kembali netral ke keadaan NULL.

· **Baris 298 - 314**, syntax pada blok fungsi hapusBelakangH() berfungsi mengamputasi memori data yang bermukim di ujung terbelakang tanpa memiliki penanda tail. Karena ketiadaan referensi ekor instan, pointer hapus harus dikerahkan untuk melakukan iterasi pencarian manual yang mahal dari awal hingga mendeteksi node yang merujuk pada NULL. Setelah posisi ujung berhasil didapat, program terlebih dahulu harus menetralkan referensi maju (next) dari node tepat sebelum sasaran (hapus->prev->next = NULL) agar rantai datanya dapat ditutup dengan rapi sebelum objek sasarannya benar-benar dihancurkan dengan perintah delete.

· **Baris 316 - 332**, syntax pada blok fungsi hapusBelakangHT() berfungsi mendealokasikan struktur simpul di posisi buritan dengan metode pergeseran penanda secara seketika. Memanfaatkan pointer arah prev bawaan struktur, indikator tail digeser mundur satu langkah secara langsung ke arah node pendahulunya (tail = tail->prev). Kemudian, sisa koneksi arah maju dari elemen ekor yang baru dijadikan NULL untuk memutuskan rantai, dan memori fisik ekor yang tertinggal dieksekusi pembebasannya.

· **Baris 334 - 343**, syntax pada blok fungsi clearH() berfungsi melancarkan operasi garbage collection menyeluruh (membumihanguskan semua rentetan memori data yang tersisa hingga tuntas tanpa jejak). Fungsi ini memanfaatkan metode iterasi pointer-chasing di mana program akan terus melompat ke elemen selanjutnya dari depan, dan secara simultan melancarkan perintah delete ke memori elemen di posisinya saat itu. Setelan initH() di tahap akhir berfungsi sebagai proklamasi bahwa struktur list telah 100% dibersihkan.

· **Baris 345 - 355**, syntax pada blok fungsi clearHT() berfungsi menormalisasi kondisi komputer dengan membebaskan ruang-ruang memori secara ekstrem pada seluruh rute list berarsitektur ganda. Setelah iterasi pembersihan node massal yang dibarengi dengan perintah delete selesai mengeksekusi elemen memori terakhir, program memanggil rutinitas fungsi initHT() yang mereset pengikat koordinat head dan tail pada titik nol dasar, menjamin program kembali fitrah seperti saat baru di-compile.

## SOAL 2

Apa fungsi next pada coding?

**Pembahasan**

Fungsi pointer next (yang dideklarasikan sebagai Tnode *next; di dalam struct) pada dasarnya adalah untuk menyimpan alamat memori dari node atau elemen data selanjutnya. Ini sangat diperlukan karena pada struktur Linked List, posisi alokasi data di dalam memori komputer letaknya tersebar dan tidak berjejer rapi seperti pada struktur Array. Di dalam kode program, pointer next berperan sebagai jembatan penghubung agar program bisa menelusuri dan membaca rentetan data dari depan ke arah belakang. Misalnya pada fungsi tampilkanH(), terdapat perulangan while(bantu != NULL) yang menjalankan perintah bantu = bantu->next;. Perintah ini bertugas pointer next digunakan sebagai acuan untuk mencari posisi data yang paling akhir, contohnya pada fungsi tambahBelakangH() dan hapusBelakangH() melalui syarat pengecekan while(bantu->next != NULL). Syarat logika ini dibuat agar penelusuran berhenti tepat di elemen terakhir dan tidak terlewat ke ruang kosong. Oleh karena itu, jika pointer next ini tidak dihubungkan dengan benar—terutama jika tidak diakhiri dengan nilai NULL pada data paling ujung—maka program akan mengalami error atau perulangan tanpa henti (infinite loop) karena sistem terus berusaha mencari lanjutan data ke alamat memori yang tidak valid.

# SOAL 3

Apa fungsi prev pada coding?

**Pembahasan**

Fungsi pointer prev (yang dideklarasikan sebagai Tnode *prev; di dalam struktur data) merupakan komponen fundamental yang mengubah sebuah Single Linked List biasa menjadi Double Linked List. Secara spesifik, pointer ini bertugas menyimpan referensi alamat memori dari elemen (node) yang posisinya berada tepat satu langkah di belakangnya (sebelumnya). Keberadaan pointer prev ini memberikan keunggulan komputasi yang besar berupa fleksibilitas penelusuran dua arah . Berdasarkan implementasi kode tersebut, peran penting prev sangat terlihat pada operasi modifikasi rentetan data. Misalnya, pada fungsi tambahDepanH(), terdapat perintah instruksi head->prev = baru;. Perintah ini berfungsi untuk menyambung kembali rantai memori ke arah mundur, di mana elemen head yang lama diinstruksi untuk mengenali dan menunjuk ke arah elemen baru yang baru saja disisipkan di depannya. hapusBelakangHT(). Pada Linked List satu arah, menghapus data paling belakang membuat program melakukan perulangan melelahkan dari awal untuk mencari elemen kedua dari belakang. Tapi karena ada referensi prev, program dapat memangkas proses tersebut menjadi sangat instan cukup dengan mengeksekusi perintah tail = tail->prev;. Perintah ini secara otomatis menggeser penanda tail mundur satu langkah, lalu memutus rantai arah majunya, sesaat sebelum men delete objek memori elemen yang paling belakang tersebut. Karena sifatnya yang krusial/penting, penunjuk arah mundur ini harus dikelola dengan teliti, pointer prev pada elemen paling head akan selalu diatur agar bernilai NULL sebagai tanda absolut pembatas awal bahwa tidak ada lagi objek memori apa pun yang mendahuluinya.

# SOAL 4

Gantilah baris 244 dan 256 dari cout<data<<' '; menjadi cout<data<<' '; lalu jawab pertanyaan berikut :

A. Apa yang terjadi jika anda menambahkan beberapa data pada program lalu tampilkan datanya, dan screenshoot hasilnya.

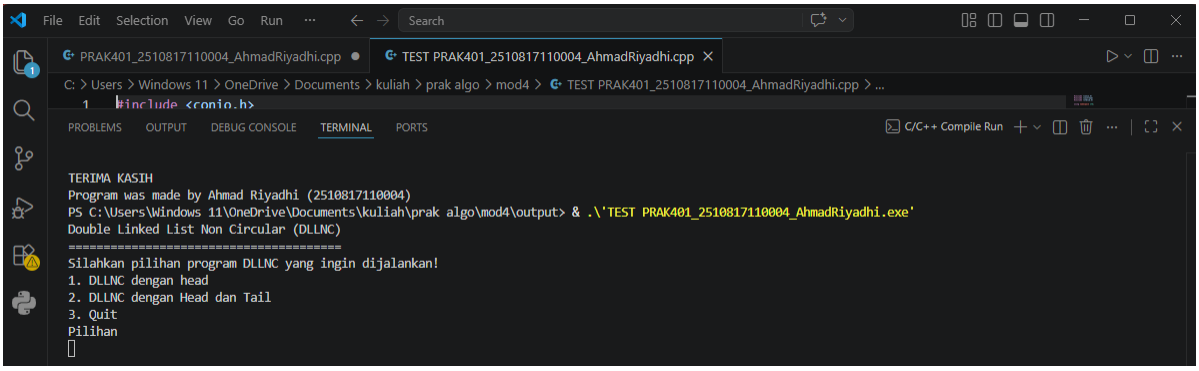
B. Jelaskan mengapa hal tersebut bisa terjadi dan data apa yang ditampilkan oleh program?

**Pembahasan**

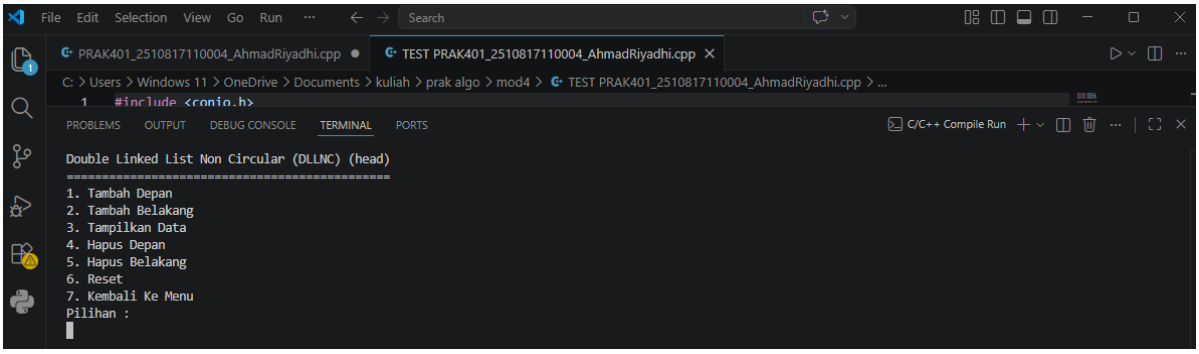
**A. Apa yang terjadi jika menambahkan beberapa data lalu tampilkan datanya?**

Jika menambahkan beberapa data secara berurutan ke dalam program (misalnya memasukkan data: "Satu", "Dua", "Tiga") lalu menjalankan menu "Tampilkan Data", program tidak akan mengalami *error* atau *crash*. Namun, program tidak mencetak data tersebut secara berurutan, melainkan hanya akan **mencetak data yang pertama kali dimasukkan secara berulang-ulang** sebanyak jumlah total data yang ada di dalam *Linked List* (contoh output menjadi: Satu Satu Satu).

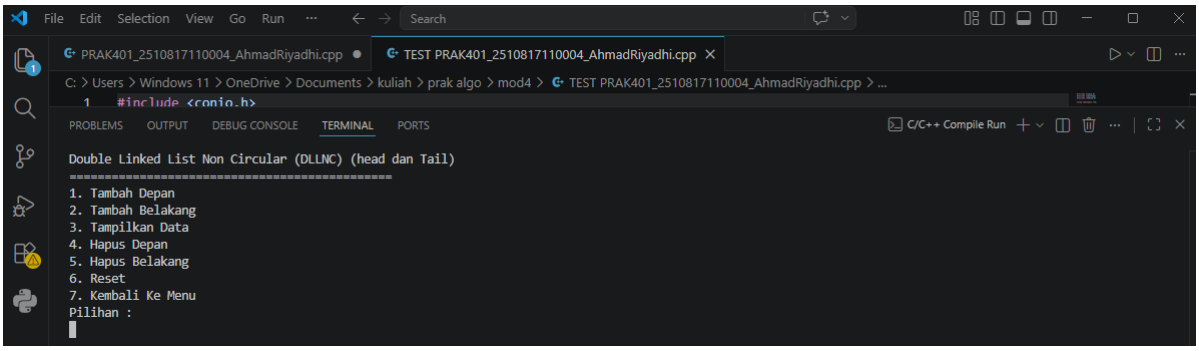
Output Program



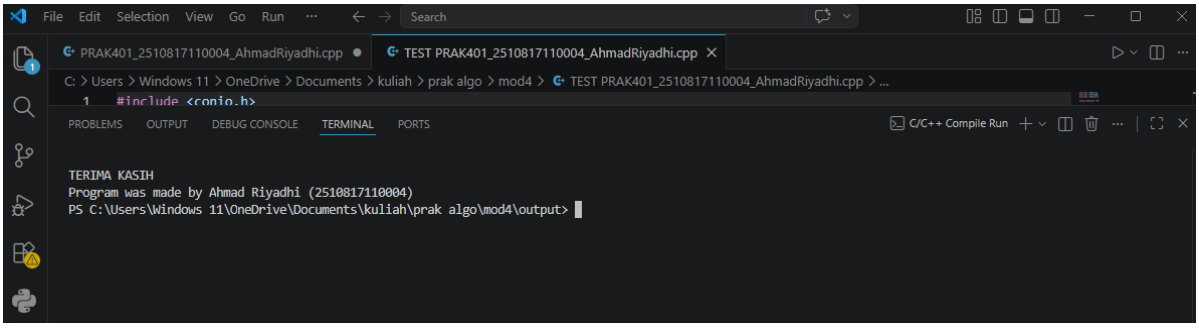

*gambar 5 Tampilan awal program soal 4*




*gambar 6 Tampilan program jika user meng input 1 2 3*




*gambar 7 Tampilan program  jika user meng input 2 3 1*




*gambar 8 Tampilan program  jika user meng input 3 2 1*

**B. Jelaskan mengapa hal tersebut bisa terjadi dan data apa yang ditampilkan oleh program?** Hal tersebut terjadi karena adanya kesalahan logika pemanggilan variabel pada perintah *output*. Pada fungsi untuk menampilkan data, program menggunakan perulangan while dengan mengandalkan pointer bantu untuk menelusuri *node* satu per satu hingga akhir list (melalui perintah bantu = bantu->next;).

Namun, karena perintah pencetakan layar diubah menjadi statis yaitu cout << head->data << ' ';, program mengabaikan pergerakan pointer bantu tersebut. Mengingat pointer head merupakan penanda pangkal yang selalu menunjuk pada *node* terdepan, maka **data yang ditampilkan oleh program hanyalah data yang berada pada urutan paling awal (pertama)** secara berulang-ulang hingga siklus perulangan while selesai.

TAUTAN GIT HUB

https://github.com/TebangLiar/Praktikum-AlgoritmaDanStrukturData-P1/tree/main/Modul%204